

COMP3308 Week 4: Game Playing

Kenny Yu

2026 Semester 1

Contents

1	Games of Perfect Information	2
1.1	Basic formal model	2
1.2	Why Greedy Search fails?	3
1.3	Minimax	3
1.4	Alpha-Beta Pruning	5
2	Games of Imperfect Information	7
2.1	Depth-Limited Search and Heuristic Evaluation	7
2.2	Evaluation functions	7
2.3	Horizon effect and quiescence	8
3	Games of Chance and Expectiminimax	9
4	Summary Results	10

1 Games of Perfect Information

1.1 Basic formal model

A **two-player zero-sum deterministic perfect-information game** is a tuple

$$G = (S, s_0, \text{Player}, A, \text{Result}, T, U),$$

where:

- S is the set of states.
- $s_0 \in S$ is the initial state.
- $\text{Player}(s) \in \{\text{MAX}, \text{MIN}\}$ specifies whose turn it is at state s .
- $A(s)$ is the set of legal actions available at state s .
- $\text{Result}(s, a)$ is the successor state obtained by applying action $a \in A(s)$.
- $T \subseteq S$ is the set of terminal states.
- $U : T \rightarrow \mathbb{R}$.

The game is:

- **deterministic** if $\text{Result}(s, a)$ is uniquely determined,
- **perfect-information** if all players observe the full state,
- **zero-sum** if MAX's gain is MIN's loss, so utilities are measured from MAX's perspective.

Definition 1.1.1 (Utility function). *The **utility function** assigns a numerical payoff to each terminal state:*

$$U(s) = \begin{cases} > 0 & \text{if } s \text{ is favourable to MAX,} \\ < 0 & \text{if } s \text{ is favourable to MIN,} \\ 0 & \text{if } s \text{ is neutral/draw.} \end{cases}$$

A larger value is always better for MAX.

Definition 1.1.2 (Evaluation function). *An **evaluation function** is a heuristic*

$$e : S \setminus T \rightarrow \mathbb{R}$$

that estimates how favourable a non-terminal state is for MAX.

Interpretation:

$$e(s) > 0 \implies s \text{ favours MAX,} \quad e(s) < 0 \implies s \text{ favours MIN.}$$

More generally, larger $e(s)$ indicates a better state for MAX.

Definition 1.1.3 (Game tree). The **game tree** rooted at state s is the tree whose:

- root is s ,
- children of a node s' are $\text{Result}(s', a)$ for $a \in A(s')$,
- leaves are terminal states, or cutoff states in DLS.

This tree represents all possible future play sequences.

1.2 Why Greedy Search fails?

A **greedy strategy** selects the move leading to the child state with the best immediate evaluation:

- MAX chooses child with largest $e(s')$,
- MIN chooses child with smallest $e(s')$.

Proposition 1.2.1. Greedy search does not in general produce the optimal move in an adversarial game.

Proof. Greedy search optimises only the immediate successor evaluation and ignores the opponent's future response. In an adversarial setting, the opponent chooses moves that are worst for the current player. Hence a move that looks best locally may allow the opponent to force a poor terminal outcome.

1.3 Minimax

Definition 1.3.1 (Minimax value). For a state s , the **minimax value** $V(s)$ is defined recursively by

$$V(s) = \begin{cases} U(s) & \text{for } s \in T, \\ \max\{V(\text{Result}(s, a)) : a \in A(s)\}, & \text{if } \text{Player}(s) = \text{MAX}, \\ \min\{V(\text{Result}(s, a)) : a \in A(s)\}, & \text{if } \text{Player}(s) = \text{MIN}. \end{cases}$$

This formalises the assumption that both players play optimally.

Theorem 1.3.1 (Correctness of minimax). In a two-player deterministic perfect-information zero-sum game, if both players play optimally, then the action selected from the root by minimax yields the best achievable outcome for the player to move.

Proof. We prove by induction on subtree depth.

- Base case: If s is terminal, then $V(s) = U(s)$, so the value is exact.

- Inductive hypothesis: Assume the theorem holds for all states at depth at most d .
- Inductive step: Consider a state s at depth $d + 1$.
 - If $\text{Player}(s) = \text{MAX}$, then MAX may choose any action $a \in A(s)$. By the inductive hypothesis, each successor $\text{Result}(s, a)$ has correct value $V(\text{Result}(s, a))$, assuming optimal play thereafter. Hence the best achievable outcome for MAX is the maximum of these values:

$$V(s) = \max\{V(\text{Result}(s, a)) : a \in A(s)\}.$$

- If $\text{Player}(s) = \text{MIN}$, then MIN chooses the successor minimising MAX's eventual outcome. Again by the induction hypothesis,

$$V(s) = \min\{V(\text{Result}(s, a))\}.$$

Thus, $V(s)$ equals the outcome forced by optimal play from both sides. Applying this to the root proves the theorem.

Corollary 1.3.1. *If MAX chooses an action*

$$a^* = \underset{a \in A(s_0)}{\text{argmax}} V(\text{Result}(s_0, a)),$$

then MAX secures the highest worst-case payoff against an optimal MIN.

Lemma 1.3.1. *If MIN does not play optimally, MAX's realised payoff cannot be worse than the minimax value.*

Proof. Let s be a MAX node and suppose MAX chooses a minimax-optimal action. By definition, the minimax value at the next MIN node is

$$\min\{V(\text{Result}(s', a)) : a \in A(s')\}.$$

If MIN deviates from the minimising move, it chooses some action with value at least this minimum. Hence the resulting outcome is no smaller than the minimax value. Therefore suboptimal MIN play can only improve MAX's actual outcome relative to the minimax guarantee.

Algorithm 1.3.1.

$$\text{MinimaxDecision}(s) = \underset{a \in A(s)}{\text{argmax}} \text{MinValue}(\text{Result}(s, a))$$

for a MAX root, where

$$\text{MaxValue}(s) = \begin{cases} U(s) & \text{for } s \in T, \\ \max\{\text{MinValue}(\text{Result}(s, a)) : a \in A(s)\} & \text{otherwise.} \end{cases}$$

and

$$\text{MinValue}(s) = \begin{cases} U(s) & \text{for } s \in T, \\ \min\{\text{MinValue}(\text{Result}(s, a)) : a \in A(s)\} & \text{otherwise.} \end{cases}$$

Proposition 1.3.1 (Complexity of minimax). *If the branching factor is b and the maximum search depth is m , then minimax has time complexity of $\Theta(b^m)$ and space complexity of $\Theta(bm)$ under depth-first implementation.*

A full depth- m tree contains exponentially many nodes, on the order of b^m . With DFS, only one root-to-leaf path plus remaining siblings must be stored, requiring linear space in depth times branching factor.

1.4 Alpha-Beta Pruning

Definition 1.4.1 (Alpha and beta bounds). *During depth-first traversal:*

- α is the best value found so far for MAX along the current path,
- β is the best value found so far for MIN along the current path.

Initially, $\alpha = -\infty$, $\beta = +\infty$.

Definition 1.4.2 (Cut-off conditions). *At node:*

- **Beta cutoff:** at a MAX node, if its current value becomes $v \geq \beta$, further exploration can stop.
- **Alpha cutoff:** at a MIN node, if its current value becomes $v \leq \alpha$, further exploration can stop.

Lemma 1.4.1 (Soundness of beta cutoff). *If at a MAX node n we discover a child value $v \geq \beta$, then the remaining children of n need not be explored.*

Proof. The node n is a descendant of some MIN ancestor whose current best alternative is bounded above by β . Since MAX can already force value at least $v \geq \beta$ through n , MIN at the ancestor will never allow play to continue into any sibling alternative whose value is at least β ; it already has an option no worse than β . Therefore the unexplored descendants of n cannot affect the final root decision.

Lemma 1.4.2 (Soundness of alpha cutoff). *If at a MIN node n we discover a child value $v \leq \alpha$, then the remaining children of n need not be explored.*

Proof. Symmetric to Lemma 1.4.1. Since MAX already has an ancestor alternative worth at least α , and MIN can force value at most $v \leq \alpha$ through node n , MAX will never choose a branch leading to n . Thus the remaining children of n cannot influence the root decision.

Theorem 1.4.1 (Correctness of alpha-beta pruning). *Alpha-beta pruning returns the same root minimax value as full minimax.*

Proof. By Lemmas 1.4.1 and 1.4.2, any pruned subtree cannot alter the backed-up value of any ancestor relevant to the final decision. Thus pruning removes only branches that are provably irrelevant to the minimax choice. Therefore the value returned at the root is unchanged from full minimax.

Algorithm 1.4.1 (Alpha-Beta Search). *For a MAX root:*

$$\text{AlphaBetaSearch}(s) = \underset{a \in A(s)}{\operatorname{argmax}} \text{MinValue}(\text{Result}(s, a), -\infty, \infty).$$

Recursive definitions:

$$\text{MaxValue}(s, \alpha, \beta) = \begin{cases} U(s) & \text{for } s \in T, \\ \max\{\text{MinValue}(\text{Result}(s, a), \alpha, \beta) : a \in A(s)\} & \text{with pruning.} \end{cases}$$

After each child:

$$\alpha \leftarrow \max\{\alpha, v\}.$$

If $v \geq \beta$, return v .

Similarly,

$$\text{MinValue}(s, \alpha, \beta) = \begin{cases} U(s) & \text{for } s \in T, \\ \min\{\text{MinValue}(\text{Result}(s, a), \alpha, \beta) : a \in A(s)\} & \text{with pruning.} \end{cases}$$

After each child:

$$\beta \leftarrow \min\{\beta, v\}.$$

If $v \leq \alpha$, return v .

Proposition 1.4.1 (Complexity of alpha-beta). *Let branching factor be b and depth be d .*

- *Worst case: $\Theta(b^d)$, same as minimax.*
- *Best case with perfect move ordering: $\Theta(b^{d/2})$.*

Thus, alpha-beta can effectively double the search depth achievable in the same time budget.

2 Games of Imperfect Information

2.1 Depth-Limited Search and Heuristic Evaluation

Definition 2.1.1 (Cutoff test). A **cutoff test** is a predicate $\text{Cutoff}(s, \ell)$ that returns true when search should stop at state s , usually because a fixed depth limit ℓ has been reached.

Instead of

if $\text{Terminal}(s)$ then return $U(s)$,

we use

if $\text{Cutoff}(s, \ell)$ then return $e(s)$.

Definition 2.1.2 (Depth-limited minimax / alpha-beta). *Depth-limited adversarial search replaces terminal utility with heuristic evaluation at cutoff leaves:*

$$V_\ell(s) = \begin{cases} U(s) & \text{for } s \in T, \\ e(s) & \text{if } \text{Cutoff}(s, \ell), \\ \max\{V_\ell(\text{Result}(s, a)) : a \in A(s)\} & \text{Player}(s) = \text{MAX}, \\ \min\{V_\ell(\text{Result}(s, a)) : a \in A(s)\} & \text{Player}(s) = \text{MIN}. \end{cases}$$

This is how practical game-playing systems operate under time limits.

Definition 2.1.3 (Iterative deepening). **Iterative deepening** performs repeated DLS for depths $1, 2, 3, \dots$ until time runs out, then returns the move from the deepest completed search.

2.2 Evaluation functions

Definition 2.2.1 (Linear evaluation function). A common evaluation function has the weighted linear form

$$\text{EVAL}(s) = \sum_{i=1}^n w_i f_i(s),$$

where $f_i(s)$ are features of the state, $w_i \in \mathbb{R}$ are corresponding weights.

For chess, examples of features include material count, queen difference, pawn structure, king safety, and mobility.

Proposition 2.2.1. *For deterministic minimax search, only the **relative ordering** induced by the evaluation function matters; exact numerical values do not.*

Proof. Minimax and alpha-beta depend only on repeated applications of max and min. If g is a strictly increasing function, then for any set of values $\{x_i\}$,

$$\underset{i}{\operatorname{argmax}} x_i = \underset{i}{\operatorname{argmax}} g(x_i), \quad \underset{i}{\operatorname{argmin}} x_i = \underset{i}{\operatorname{argmin}} g(x_i).$$

Hence replacing e by $g \circ e$ preserves all comparisons and therefore preserves the selected move.

Corollary 2.2.1. *Any monotone increasing transformation of the evaluation function preserves minimax behaviour in deterministic games.*

Although linear evaluation is convenient, it may be inaccurate because features can interact. For example, the value of a bishop or pawn may depend on board structure and game phase, violating simple independence assumptions.

2.3 Horizon effect and quiescence

Definition 2.3.1 (Horizon effect). *The **horizon effect** occurs when a damaging event lies just beyond the search depth limit, so the evaluation function fails to detect it.*

Definition 2.3.2 (Quiescent and non-quiescent states). *A state is **quiescent** if it is unlikely to change drastically in the near future. A state is **non-quiescent** if a tactical event such as a capture, promotion, or immediate threat is imminent.*

Proposition 2.3.1. *Applying the evaluation function at non-quiescent states can produce misleading search decisions.*

At such states, the heuristic estimate may fail to account for imminent sharp tactical swings. Therefore a shallow cutoff can mis-rank moves.

Algorithm 2.3.1 (Quiescence extension, conceptual). *Procedure:*

1. *Perform depth-limited minimax or alpha-beta.*
2. *If a leaf state at the cutoff depth is non-quiescent, extend search further.*
3. *Restrict this extension to tactically critical moves such as captures, promotions, or checks.*
4. *Evaluate only once a quiescent state is reached.*

3 Games of Chance and Expectiminimax

Definition 3.0.1 (Game with chance nodes). *A game with chance contains, in addition to MAX and MIN nodes, **chance nodes** representing random events such as dice rolls. Each outgoing branch i from a chance node has probability p_i , with*

$$p_i \geq 0, \quad \sum_i p_i = 1.$$

Definition 3.0.2 (Expectiminimax value). *For a game with chance, the **expectiminimax value** $V(s)$ is defined by*

$$V(s) = \begin{cases} U(s) & \text{for } s \in T, \\ \max\{V(\text{Result}(s, a)) : a \in A(s)\} & \text{if } \text{Player}(s) = \text{MAX}, \\ \min\{V(\text{Result}(s, a)) : a \in A(s)\} & \text{if } \text{Player}(s) = \text{MIN}, \\ \sum_i p_i V(s_i) & s \text{ is a chance node with children } s_i. \end{cases}$$

Theorem 3.0.1 (Correctness of expectiminimax). *In a perfect-information game with chance, expectiminimax computes the optimal decision under rational play and known outcome probabilities.*

Proof sketch. The proof is by induction on tree depth.

- Terminal nodes use exact utility.
- MAX nodes choose the action with maximum expected continuation value.
- MIN nodes choose the action with minimum expected continuation value.
- Chance nodes take expectation under the probability law of the random event.

By the law of total expectation and optimal play at decision nodes, the recursive definition yields the correct expected utility from every state.

Example 3.0.1. If a chance node has two equally likely children with utilities 2 and 4, then

$$V = 0.5 \times 2 + 0.5 \times 4 = 3.$$

Proposition 3.0.1 (Complexity of expectiminimax). *If b is the move branching factor, m is depth, and n is the number of distinct chance outcomes per chance node, then expectiminimax is more expensive than minimax because it must also branch over random outcomes.*

Proposition 3.0.2. *For games with chance, exact evaluation values matter, not merely their relative ordering.*

Proof. At chance nodes, decisions depend on expected values:

$$E(X) = \sum_i p_i x_i.$$

A monotone nonlinear transformation g does not in general preserve expectations:

$$\sum_i p_i g(x_i) \neq g\left(\sum_i p_i x_i\right).$$

Therefore two evaluation scales with identical orderings can induce different expected values and hence different decisions.

4 Summary Results

Theorem 4.0.1 (Main summary theorem). *For two-player zero-sum perfect-information games:*

1. **Minimax** returns the optimal move against an optimal opponent.
2. **Alpha-beta pruning** returns the same move as minimax while potentially evaluating many fewer nodes.
3. **Depth-Limited Search** with an evaluation function makes adversarial search practical under time limits.
4. **Expectiminimax** extends minimax to games with chance by replacing deterministic backup at chance nodes with expectation.